МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

з навчальної дисципліни «Професійний Python»

Варіант 4: «Система банківських рахунків»

Виконав:
студент групи ФЕП-33
Зінчук Остап

Викладач:
Проф. Свелеба Сергій Андрійович

Львів — 2026

1. Мета роботи

Мета лабораторної роботи: Опанувати теоретичні засади та практичні інженерні навички професійного автоматизованого тестування програмного забезпечення мовою Python із застосуванням тестового фреймворку pytest, навчитися проєктувати модульні (unit) та інтеграційні (integration) тести, створювати фікстури, застосовувати параметризацію, перевіряти винятки через pytest.raises, ізолювати файлову систему (tmp_path) та змінні оточення (monkeypatch), застосовувати тестові дублери (Mock, AsyncMock, Fake) та здійснювати всебічний аудит покриття коду розгалуженнями (Branch Coverage).

2. Постановка завдання за Варіантом 4

Згідно з індивідуальним Варіантом 4 («Система банківських рахунків») об'єктом тестування є комплексний банківський модуль управління фінансовими рахунками. Програма тестування охоплює такі ключові вимоги:

- Створення сутності Account та валідація номера рахунку, імені клієнта і початкового балансу;
- Операції поповнення (deposit) та зняття коштів (withdraw);
- Заборона від'ємного балансу та викидання InsufficientFundsError при нестачі коштів;
- Пошук рахунку в репозиторії, перевірка поточного балансу та обчислення загальної суми коштів;
- Граничне тестування (нульовий залишок, мікрокопійки) та відхилення невалідних сум через InvalidAmountError;
- Імпорт рахунків з CSV-файлів (import_accounts_csv) та обробка пошкоджених записів (DataImportError);
- Експорт рахунків у JSON (export_accounts_json) з повною перевіркою файлової системи;
- Конфігурація дозволених операцій та валюти з тестуванням змінних оточення через monkeypatch;
- Тестування репозиторію через Mock та Fake-реалізацію (FakeAccountRepository);
- Тестування клієнтського сповіщувача через Mock та моделювання збоїв за допомогою side_effect;
- Тестування зовнішньої банківської залежності через AsyncMock та наскрізний потік операцій (Account Flow).

3. Структура проєкту (суто згідно з методичкою)

Архітектура проєкту побудована у суворій відповідності до структури, наведеної у розділі 8 та прикладі розділу 58 методички:

```
Lab_06/
├── pyproject.toml
├── setup.cfg
├── README.md
├── src/
│   └── bank_system/
│       ├── __init__.py
│       ├── models.py
│       ├── protocols.py
│       ├── repositories.py
│       ├── services.py
│       ├── config.py
│       ├── exceptions.py
│       └── exporters.py
└── tests/
    ├── conftest.py
    ├── unit/
    │   ├── test_models.py
    │   ├── test_services.py
    │   ├── test_configuration.py
    │   └── test_exporters.py
    └── integration/
        ├── test_import_export.py
        └── test_account_flow.py
```

4. Тестова матриця (Test Matrix)

Матриця покриває всі вимоги розділу 91 методички: нормальні, граничні, некоректні дані, відмови залежностей та інтеграцію.

Компонент	Тестований сценарій	Тип тесту	Очікуваний результат
Account	Створення з валідними даними	Normal Unit	Об'єкт створено, атрибути співпадають
Account	Порожній номер або ім'я клієнта	Invalid Unit	AccountValidationError
Account	Від'ємний початковий баланс	Boundary/Invalid	AccountValidationError
Account	Deposit додатної суми	Normal Unit	Баланс збільшено, pytest.approx
Account	Deposit недопустимої суми (<= 0)	Boundary/Invalid	InvalidAmountError
Account	Withdraw коректної суми	Normal Unit	Баланс зменшено
Account	Withdraw всього залишку (під нуль)	Boundary Unit	Баланс стає рівним 0.00
Account	Withdraw більше балансу	Failure Unit	InsufficientFundsError
BankingService	Deposit з викликом Notifier	Mock Interaction	assert_called_once_with
BankingService	Збій Notifier через side_effect	Failure Path	NotificationError перехоплено
BankingService	Переказ між рахунками (transfer)	State/Interaction	Обидва баланси оновлено, SMS надіслано

AsyncBankingGateway	Асинхронна авторизація шлюзом	AsyncMock Unit	assert_awaited_once_with
Config	Читання BANK_CURRENCY з оточення	Monkeypatch Unit	monkeypatch.setenv/delenv працює
Exporters	Імпорт CSV та експорт JSON	tmp_path Unit	Файли створено в пісочниці tmp_path
Pipeline	CSV -> обробка -> JSON	Integration	Наскрізний потік даних без помилок
Account Flow	Створення -> шлюз -> транзакції -> JSON	Integration E2E	Повна цілісність системи

5. Реалізація тестів та інструментів тестування

5.1. Фікстури та фабрика рахунків (conftest.py)

У tests/conftest.py реалізовано централізовані фікстури для чистого створення тестових об'єктів та патерн Fixture Factory (завдання підвищеної складності, розділ 96 методички):

```
@pytest.fixture
def account_factory() -> Callable[..., Account]:
    counter = 0
    def _create(account_number=None, client='Тест', balance=1000.0, currency='UAH') -> Account:
        nonlocal counter
        counter += 1
        acc_num = account_number or f'UA{counter:08d}'
        return Account(account_number=acc_num, client=client, balance=balance, currency=currency)
    return _create
```

5.2. Параметризація, pytest.raises та pytest.approx

Тестування відхилення невалідних сум та граничних умов з використанням параметризації та перевірки винятків:

```
@pytest.mark.parametrize(
    'invalid_amount',
    [0.0, -0.01, -100.0],
    ids=['zero_amount', 'negative_cent', 'large_negative'],
)
def test_deposit_invalid_amount_raises(sample_account: Account, invalid_amount: float) -> None:
    with pytest.raises(InvalidAmountError, match='Сума поповнення повинна бути додатною'):
        sample_account.deposit(invalid_amount)
def test_withdraw_boundary_exact_balance(sample_account: Account) -> None:
    balance_to_withdraw = sample_account.balance
    new_balance = sample_account.withdraw(balance_to_withdraw)
    assert new_balance == pytest.approx(0.0)
    assert sample_account.balance == pytest.approx(0.0)
```

5.3. Ізоляція файлової системи (tmp_path) та оточення (monkeypatch)

Використання вбудованої фікстури tmp_path для гарантії незалежності файлових тестів та monkeypatch для динамічного керування конфігурацією без побічних ефектів:

```
def test_export_accounts_json_creates_valid_file(tmp_path: Path, sample_account: Account) -> None:
    output_file = tmp_path / 'accounts_export.json'
    export_accounts_json([sample_account], output_file)
    assert output_file.exists()
    content = json.loads(output_file.read_text(encoding='utf-8'))
    assert content[0]['account_number'] == 'UA10000001'
def test_default_currency_from_environment(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.setenv('BANK_CURRENCY', 'EUR')
    assert get_default_currency() == 'EUR'
    monkeypatch.delenv('BANK_CURRENCY', raising=False)
    assert get_default_currency() == 'UAH'
```

5.4. Макетування Mock, перевірка взаємодій та моделювання збоїв (side_effect)

Ізоляція сповіщувача через Mock(spec=Notifier), верифікація викликів через assert_called_once_with та моделювання відмови поштової системи за допомогою side_effect:

```
def test_notifier_failure_side_effect(populated_repository: FakeAccountRepository) -> None:
    failing_notifier = Mock(spec=Notifier)
    failing_notifier.send.side_effect = NotificationError('Помилка доставки SMS')
    service = BankingService(repository=populated_repository, notifier=failing_notifier)
    with pytest.raises(NotificationError, match='Помилка доставки SMS'):
        service.process_deposit('UA00000001', 200.0, 'test@example.com')
    failing_notifier.send.assert_called_once()
```

5.5. Асинхронне тестування через AsyncMock

Тестування взаємодії з асинхронним зовнішнім банківським шлюзом за допомогою AsyncMock та верифікація виклику корутини через assert_awaited_once_with:

```
@pytest.mark.asyncio
async def test_async_banking_gateway_success(mock_async_client: AsyncMock) -> None:
    gateway = AsyncBankingGateway(client=mock_async_client)
    status = await gateway.verify_transaction('UA00000001', 500.0)
    assert status == 'APPROVED'
    mock_async_client.authorize.assert_awaited_once_with('UA00000001', 500.0)
```

5.6. Інтеграційні тести (Integration Pipeline & Account Flow)

Інтеграційні тести перевіряють злагоджену взаємодію компонентів у наскрізному процесі:

```
@pytest.mark.integration
def test_csv_to_json_full_pipeline(tmp_path: Path) -> None:
    csv_file = tmp_path / 'incoming.csv'
    csv_file.write_text('account_number,client,balance,currency\nUA1,Остап,1000.0,UAH\n')
    accounts = import_accounts_csv(csv_file)
    accounts[0].deposit(500.0)
    out_json = tmp_path / 'out.json'
    export_accounts_json(accounts, out_json)
    data = json.loads(out_json.read_text(encoding='utf-8'))
    assert data[0]['balance'] == pytest.approx(1500.0)
```

6. Результати виконання тестів та аналіз покриття (Coverage Report)

Запуск повного набору тестів виконано за допомогою офіційної команди з методички (розділ 82):
pytest -v --cov=bank_system --cov-branch --cov-report=term-missing --cov-report=html

```
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\University\year3\Професійний Python\python-labs\Lab_06
configfile: pyproject.toml
plugins: asyncio-1.4.1.post1, cov-7.1.0
asyncio: mode=Mode.AUTO
collected 44 items
tests/integration/test_account_flow.py::test_full_account_operation_flow PASSED [ 2%]
tests/integration/test_account_flow.py::test_atomic_transfer_failure_preserves_balances PASSED [ 4%]
tests/integration/test_import_export.py::test_csv_to_json_full_pipeline PASSED [ 6%]
tests/integration/test_import_export.py::test_empty_csv_import_and_export PASSED [ 9%]
tests/unit/test_configuration.py (5 tests) ............................. PASSED [ 20%]
tests/unit/test_exporters.py (7 tests) ................................. PASSED [ 36%]
tests/unit/test_models.py (16 tests) ................................... PASSED [ 72%]
tests/unit/test_services.py (12 tests) ................................. PASSED [100%]
================================ tests coverage ================================
Name                                Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------
src\bank_system\__init__.py             8      0      0      0   100%
src\bank_system\config.py              13      0      0      0   100%
src\bank_system\exceptions.py           9      0      0      0   100%
src\bank_system\exporters.py           44      4     10      1    91%   21, 54-55, 62
src\bank_system\models.py              36      1     12      0    98%   77
src\bank_system\protocols.py           11      0      0      0   100%
src\bank_system\repositories.py        17      0      0      0   100%
src\bank_system\services.py            49      2     12      2    93%   48, 94
-------------------------------------------------------------------------------
TOTAL                                 187      7     34      3    95.48%
Required test coverage of 80.0% reached. Total coverage: 95.48%
============================== 44 passed in 0.37s ==============================
```

Аналіз метрик:

• Загальна кількість тестів: 44 (40 модульних, 4 інтеграційних). Усі 44 пройшли успішно (100% pass rate).

• Швидкість виконання suite: 0.37 секунди (висока швидкодія завдяки ізоляції та детермінізму).

• Покриття розгалужень (Branch Coverage): 95.48% (вимога методички >= 80% перевиконана на 15.48%).

• Непокриті шляхи: 7 рядків стосуються рідкісних винятків файлової системи (захист диску від запису, помилка читання заголовків) та захисних гілок, що є природним для надійних продакшн-систем.

7. Відповіді на контрольні запитання

Нижче наведено вичерпні академічні відповіді на всі 80 контрольних запитань розділу 106 методички:

1. Що таке автоматизоване тестування?

Відповідь: Це процес верифікації програмного забезпечення за допомогою спеціальних програмних сценаріїв (тестів), які автоматично запускають код, подають вхідні дані та перевіряють відповідність фактичного результату очікуваному без втручання людини.

2. Чим автоматизовані tests відрізняються від ручного тестування?

Відповідь: Автоматизовані тести виконуються багаторазово за мілісекунди, повністю відтворювані, виключають людський фактор неуважності, інтегруються в CI/CD та дозволяють безпечно проводити масштабний рефакторинг.

3. Що таке регресія (regression)?

Відповідь: Це поява помилки у вже протестованій та працездатній функціональності системи внаслідок внесення нових змін, додавання функцій або виправлення інших дефектів.

4. Чому ручне тестування неефективне для запобігання regressions?

Відповідь: З ростом проєкту кількість тестових сценаріїв зростає комбінаторно; ручна перевірка всіх сценаріїв після кожної зміни вимагає колосального часу, є надмірно дорогою і схильною до людських помилок.

5. Що таке test pyramid?

Відповідь: Це концептуальна модель організації тестового набору, яка стверджує, що базу мають складати швидкі й ізольовані unit-тести (70-80%), середній рівень — інтеграційні тести (15-20%), а вершину — нечисленні й дорогі e2e/UI-тести (5-10%).

6. Чому unit tests мають складати основу піраміди?

Відповідь: Unit-тести виконуються за частки мілісекунди, точно локалізують місце виникнення дефекту, не залежать від зовнішніх сервісів, є дешевими в написанні та стабільними у підтримці.

7. Чим unit test відрізняється від integration test?

Відповідь: Unit-тест перевіряє поведінку окремого ізольованого компонента (класу чи функції) без зовнішніх залежностей; integration test перевіряє коректність взаємодії кількох модулів, підсистем, файлової системи чи баз даних.

8. Що таке pytest?

Відповідь: Це сучасний, найпопулярніший у Python тестовий фреймворк, який підтримує використання вбудованого оператора assert, потужну систему фікстур, параметризацію та багату екосистему плагінів.

9. Чому pytest популярніший за unittest?

Відповідь: Pytest усуває необхідність створювати громіздкі класи-нащадки TestCase, використовує стандартний лаконічний assert замість self.assertEqual, має значно гнучкішу систему DI-фікстур та параметризації.

10. Як pytest знаходить tests?

Відповідь: За механізмом Test Discovery: шукає файли за шаблонами test_*.py або *_test.py, усередині них знаходить тестові функції з префіксом test_* та тестові класи з префіксом Test (без методу __init__).

11. Які правила найменування test files і test functions?

Відповідь: Файли: test_.py або _test.py; функції: test__() або test_(); класи: Test.

12. Як запустити tests через команду pytest?

Відповідь: У терміналі в корені проєкту: 'pytest' для стандартного запуску, 'pytest -v' для детального, 'pytest tests/unit' для вибіркового запуску конкретної директорії.

13. Як зупинити виконання tests після першого failure?

Відповідь: Використовувати прапорець '-x' або '--exitfirst' (наприклад: 'pytest -x').

14. Що робить прапорець -v у pytest?

Відповідь: Вмикає режим verbose: виводить назву кожного тестового модуля, ім'я кожної тестової функції та індивідуальний статус виконання (PASSED, FAILED, SKIPPED).

15. Що таке патерн Arrange - Act - Assert?

Відповідь: Класичний патерн структури тесту: Arrange — підготовка початкового стану та даних; Act — виконання тестованої дії; Assert — перевірка отриманого результату.

16. Чому assert у pytest інформативніший за стандартний assert у Python?

Відповідь: Pytest використовує AST-rewriting (переписування дерева синтаксису), завдяки чому при failure виводить докладні проміжні значення виразів, різницю між рядками та зрізи колекцій.

17. Як перевірити рівність значень у pytest?

Відповідь: За допомогою стандартного порівняння: assert result == expected.

18. Чому небезпечно порівнювати floating-point через ==?

Відповідь: Через особливості представлення дробових чисел у стандарті IEEE 754 виникають похибки округлення (наприклад, 0.1 + 0.2 != 0.3).

19. Як використовувати pytest.approx?

Відповідь: Застосовувати обгортку: assert result == pytest.approx(expected, rel=1e-6, abs=1e-9).

20. Як перевірити, що function генерує exception через pytest.raises?

Відповідь: Використати контекстний менеджер: with pytest.raises(ExpectedException): function_call().

21. Як перевірити exception message?

Відповідь: Передати параметр match: with pytest.raises(MyError, match='текст шаблону regex'): ...

22. Як отримати exception object через exc_info?

Відповідь: Через змінну контексту: with pytest.raises(MyError) as exc_info: ...; далі досліджувати exc_info.value.

23. Що таке boundary testing?

Відповідь: Метод розробки тестів, що фокусується на перевірці поведінки програми на границях допустимих діапазонів (на межах, трохи нижче та трохи вище границі).

24. Які boundaries потрібно тестувати для діапазону 0–100?

Відповідь: Мінімальну границю (-1, 0, 1), внутрішні типові значення (50) та максимальну границю (99, 100, 101).

25. Що таке parametrization?

Відповідь: Техніка запуску однієї й тієї самої логіки тестової функції над різними наборами вхідних даних та очікуваних результатів без дублювання коду тесту.

26. Для чого використовується декоратор @pytest.mark.parametrize?

Відповідь: Для оголошення списку аргументів та значень, які по черзі передаватимуться у тестову функцію як окремі тестові кейси.

27. Що дають ids у параметризованих tests?

Відповідь: Зрозумілі текстові ідентифікатори в консолі та звітах, які показують суть кожного конкретного тестового випадку замість сирих значень.

28. Що таке fixture у pytest?

Відповідь: Це функція, декорована @pytest.fixture, яка забезпечує повторно використовувану підготовку середовища (Arrange) та коректне очищення ресурсів (Teardown).

29. Для чого використовуються fixtures?

Відповідь: Для усунення дублювання коду підготовки даних, забезпечення чистоти та ізоляції тестів і керування спільними ресурсами.

30. Що означає fixture setup?

Відповідь: Частина коду фікстури, що виконується до запуску тіла тесту та готує потрібні об'єкти чи стан.

31. Що означає fixture teardown?

Відповідь: Частина коду фікстури, що виконується після завершення тесту (успішного або аварійного) для звільнення пам'яті, закриття з'єднань чи видалення файлів.

32. Для чого yield у fixture?

Відповідь: Розділяє фази: код до yield виконує Setup і повертає значення у тест; код після yield гарантовано виконує Teardown.

33. Які fixture scopes підтримує pytest?

Відповідь: Підтримує scopes: function (дефолтний), class, module, package та session.

34. Який scope є default?

Відповідь: Scope 'function' — фікстура створюється заново для кожного окремого тесту, що забезпечує максимальну ізоляцію.

35. Для чого файл conftest.py?

Відповідь: Для глобального визначення спільних фікстур, хуків та плагінів, доступних усім тестам у поточному каталозі та підкаталогах без явного імпорту.

36. Чому fixtures із conftest.py не потрібно імпортувати вручну?

Відповідь: Pytest автоматично сканує conftest.py при завантаженні тестових модулів та реєструє фікстури у внутрішньому реєстрі за їхніми іменами.

37. Що таке tmp_path?

Відповідь: Вбудована фікстура pytest, яка повертає унікальний об'єкт pathlib.Path до тимчасової директорії, створеної спеціально для поточного тесту.

38. Чому tmp_path важливий для test isolation?

Відповідь: Гарантує, що файлові операції тесту не засмічують реальний диск, не впливають на паралельні тести та автоматично ізольовані.

39. Що таке test isolation?

Відповідь: Принцип, згідно з яким жоден тест не повинен залежати від результату або стану, залишеного іншими тестами, і може виконуватися в довільному порядку.

40. Для чого використовується фікстура monkeypatch?

Відповідь: Для безпечної тимчасової підміни змінних середовища, атрибутів модулів/класів, функцій чи словників на час виконання одного тесту.

41. Як monkeypatch змінює environment variable?

Відповідь: Через метод monkeypatch.setenv('KEY', 'VALUE'), який після завершення тесту автоматично повертає початковий стан.

42. Як видалити environment variable у test?

Відповідь: Через monkeypatch.delenv('KEY', raising=False).

43. Як підмінити function через monkeypatch?

Відповідь: Через monkeypatch.setattr(target_object, 'attribute_name', replacement).

44. Що таке test double?

Відповідь: Загальний термін для будь-якого фіктивного об'єкта, що підміняє реальний компонент під час тестування (Dummy, Stub, Fake, Mock, Spy).

45. Що таке Dummy?

Відповідь: Об'єкт-заглушка, який просто передається у списку параметрів, але методи якого ніколи фактично не викликаються.

46. Що таке Stub?

Відповідь: Об'єкт, який повертає жорстко закодовані готові відповіді на виклики під час тесту, не виконуючи реальної логіки.

47. Що таке Fake?

Відповідь: Об'єкт, що має робочу бізнес-імплементацію, але спрощену й не придатну для продакшну (наприклад, FakeAccountRepository на базі dict).

48. Що таке Mock?

Відповідь: Об'єкт, що записує всі виклики, які до нього надходять (аргументи, кількість), дозволяючи перевіряти протокол взаємодії через асерти.

49. Що таке Spy?

Відповідь: Реальний об'єкт або функція, загорнута у шпигунську оболонку, яка дозволяє перехоплювати та аналізувати фактичні виклики.

50. Для чого призначений unittest.mock.Mock?

Відповідь: Для створення універсальних тестових дублерів, макетування методів та перевірки протоколів міжкомпонентної взаємодії.

51. Що таке spec у Mock?

Відповідь: Обмеження інтерфейсу мока: забороняє викликати або призначати методи й атрибути, яких немає в реальному класі чи протоколі.

52. Для чого використовується return_value?

Відповідь: Визначає фіксоване значення, яке повертає мок-метод при звичайному виклику.

53. Для чого використовується side_effect?

Відповідь: Дозволяє викидати винятки при виклику, повертати послідовність значень або викликати кастомну функцію-обробник.

54. Як перевірити, що method викликано один раз із конкретними параметрами?

Відповідь: За допомогою методу mock.method.assert_called_once_with(*args, **kwargs).

55. Як перевірити parameters виклику?

Відповідь: Через mock.method.call_args або mock.method.call_args_list.

56. Як перевірити, що method не викликався?

Відповідь: За допомогою перевірки mock.method.assert_not_called().

57. Що таке state testing?

Відповідь: Підхід до тестування, за якого перевіряється кінцевий стан об'єкта чи системи після виконання операції (наприклад, зміна балансу).

58. Що таке interaction testing?

Відповідь: Підхід, за якого перевіряється факт і протокол взаємодії між компонентами (які методи викликано, з якими аргументами і скільки разів).

59. Коли interaction testing виправдане?

Відповідь: На системних границях: відправка email/SMS сповіщень, виклики платіжних шлюзів, зовнішніх API, логування.

60. Що таке unittest.mock.patch?

Відповідь: Механізм підміни реальних об'єктів або функцій за шляхом їх імпорту на моки всередині контексту або через декоратор.

61. Чому patch потрібно застосовувати там, де об'єкт використовується, а не там, де він визначений?

Відповідь: Тому що модуль, який імпортував об'єкт, посилається на локальне ім'я у своєму просторі імен; патчити треба саме це локальне посилання.

62. Що таке AsyncMock?

Відповідь: Спеціалізований мок із модуля unittest.mock, який імітує асинхронні функції (корутини) та підтримує await.

63. Як перевірити awaited call в AsyncMock?

Відповідь: Через асерти async_mock.assert_awaited_once() або async_mock.assert_awaited_once_with(*args).

64. Що таке flaky test?

Відповідь: Недетермінований тест, який без зміни коду то проходить успішно, то падає (через race conditions, таймінги, мережу чи рандом).

65. Чому flaky tests небезпечні?

Відповідь: Вони руйнують довіру до автоматизованого тестування, змушують розробників ігнорувати падіння CI та витрачати час на хибні тривоги.

66. Що таке deterministic test?

Відповідь: Тест, який за однакових умов завжди дає стабільно однаковий результат незалежно від часу запуску, платформи та оточення.

67. Чому unit test не повинен залежати від Інтернету чи зовнішніх баз даних?

Відповідь: Тому що зовнішні мережі повільні, нестабільні, можуть бути недоступними і перетворюють швидкі тести на повільні та flaky.

68. Коли в integration test доцільно використовувати real filesystem?

Відповідь: Коли тестується логіка збереження, атомарного перезапису або парсингу файлів; при цьому слід використовувати tmp_path.

69. Чому не потрібно мокувати абсолютно все?

Відповідь: Надмірне макетування призводить до надмірного зв'язування з реалізацією (over-mocking), коли тести перевіряють виклики моків замість реальної поведінки програми.

70. Що таке over-mocking?

Відповідь: Антипатерн тестування, коли всі залежності підмінено моками, внаслідок чого тест втрачає здатність виявляти реальні помилки інтеграції.

71. Що таке test coverage?

Відповідь: Метрика, яка показує відсоток вихідного коду програми (рядків або розгалужень), який було фактично виконано під час прогону тестів.

72. Що таке statement coverage?

Відповідь: Відсоток рядків коду, які були виконані хоча б один раз під час тестування.

73. Що таке branch coverage і чим воно відрізняється від statement coverage?

Відповідь: Branch coverage перевіряє виконання кожного можливого напрямку умовних переходів (if/else), виявляючи непротестовані логічні гілки навіть при 100% рядковому покритті.

74. Як запустити аналіз покриття через pytest-cov?

Відповідь: Командою: pytest --cov=bank_system --cov-branch --cov-report=term-missing.

75. Для чого потрібен прапорець --cov-report=term-missing?

Відповідь: Для виведення в консоль точних номерів рядків, які не були зачеплені жодним тестом.

76. Для чого потрібен HTML coverage report?

Відповідь: Для інтерактивної візуалізації вихідного коду у браузері з кольоровим підсвічуванням покритих (зеленим) і пропущених (червоним) рядків і розгалужень.

77. Чи гарантує 100% coverage відсутність дефектів?

Відповідь: Ні, покриття фіксує лише факт виконання рядка інтерпретатором, але не гарантує повноти перевірок assert або коректності бізнес-логіки.

78. Чому якісні meaningful assertions важливіші за високий відсоток coverage?

Відповідь: Тому що тест без асертів або зі слабкими асертами може мати 100% coverage, але пропускати абсолютно всі критичні помилки програми.

79. Що таке coverage threshold?

Відповідь: Мінімально допустимий поріг покриття (наприклад, fail_under = 80 у pyproject.toml), нижче якого прогін тестів у CI вважається провальним.

80. Чому unit tests мають бути взаємно незалежними?

Відповідь: Незалежність дозволяє запускати тести в будь-якому порядку, паралельно в кілька потоків, швидко перезапускати окремі невдалі тести та спрощує їх підтримку.

8. Висновки

Під час виконання Лабораторної роботи №6 було всебічно опановано сучасну інженерну методологію автоматизованого тестування програмного забезпечення мовою Python на прикладі системи банківських рахунків (Варіант 4) у суворій відповідності до структури та академічних вимог методичних вказівок.

Основні практичні результати та досягнення:

1. Створено лаконічну, модульну та масштабовану архітектуру банківського домену (пакет bank_system), що включає сутність Account, інтерфейсні протоколи, Fake-репозиторій, бізнес-сервіс, платіжний шлюз та модуль імпорту/експорту.
2. Реалізовано повний набір із 44 автоматизованих тестів (40 модульних та 4 інтеграційних), що бездоганно виконуються за 0.37 с.
3. Досягнуто показник покриття коду розгалуженнями (Branch Coverage) на рівні 95.48% (при мінімальній вимозі 80%).
4. Застосовано патерн Fixture Factory, параметризацію з ids, ізоляцію файлової системи через tmp_path, ізоляцію оточення через monkeypatch.
5. Реалізовано коректне макетування зовнішніх сервісів через Mock(spec=...) та AsyncMock із суворою перевіркою викликів та моделюванням збоїв (side_effect).
6. Забезпечено бездоганну якість коду: нуль зауважень лінтера flake8 та повна

сумісність із суворим режимом типізації муру.